

Página de Análisis de Datos Genéticos de Antropología Molecular

Alessandro López-Hernández

2026-01-04

Table of contents

1	Bienvenida	3
1.1	Instalación de R y RStudio	3
2	R para principiantes	5
3	Vectores	8
4	Matrices	9
5	Data frames	11
6	Listas	13
7	Trabajos en el Laboratorio I	15
7.1	1. Descarga de datos desde GenBank	15
8	Summary	18
	References	19

1 Bienvenida

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

1.1 Instalación de R y RStudio

Para comenzar, necesitamos instalar R y RStudio. R es el lenguaje de programación, mientras que RStudio es un entorno de desarrollo integrado (IDE) que facilita el uso de R. 1. **Instalar R:** Puedes descargar R desde el sitio web del [Proyecto R](#). Sigue las instrucciones para tu sistema operativo (Windows, macOS o Linux). 2. **Instalar RStudio:** Después de instalar R, puedes descargar RStudio desde el sitio web de [RStudio](#). Elige la versión gratuita (RStudio Desktop) y sigue las instrucciones de instalación.

Una vez que hayas instalado R y RStudio, abre RStudio para comenzar a trabajar con tus datos de secuencias de ADN mitocondrial. En la próxima sección, aprenderemos cómo cargar y manipular estos datos en R.

¡Bienvenidos al **Laboratorio de Antropología Molecular** de la Facultad de Ciencias, UNAM!.



Figure 1.1: Miembros del Laboratorio de AM en Teotihuacán. Créditos a Alessandro López-Hernandez, 2023.

2 R para principiantes

Nota: esta sección está diseñada para estudiantes que no tienen experiencia previa en programación, en especial a R. Si ya tienes experiencia en programación, puedes saltar esta sección.

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

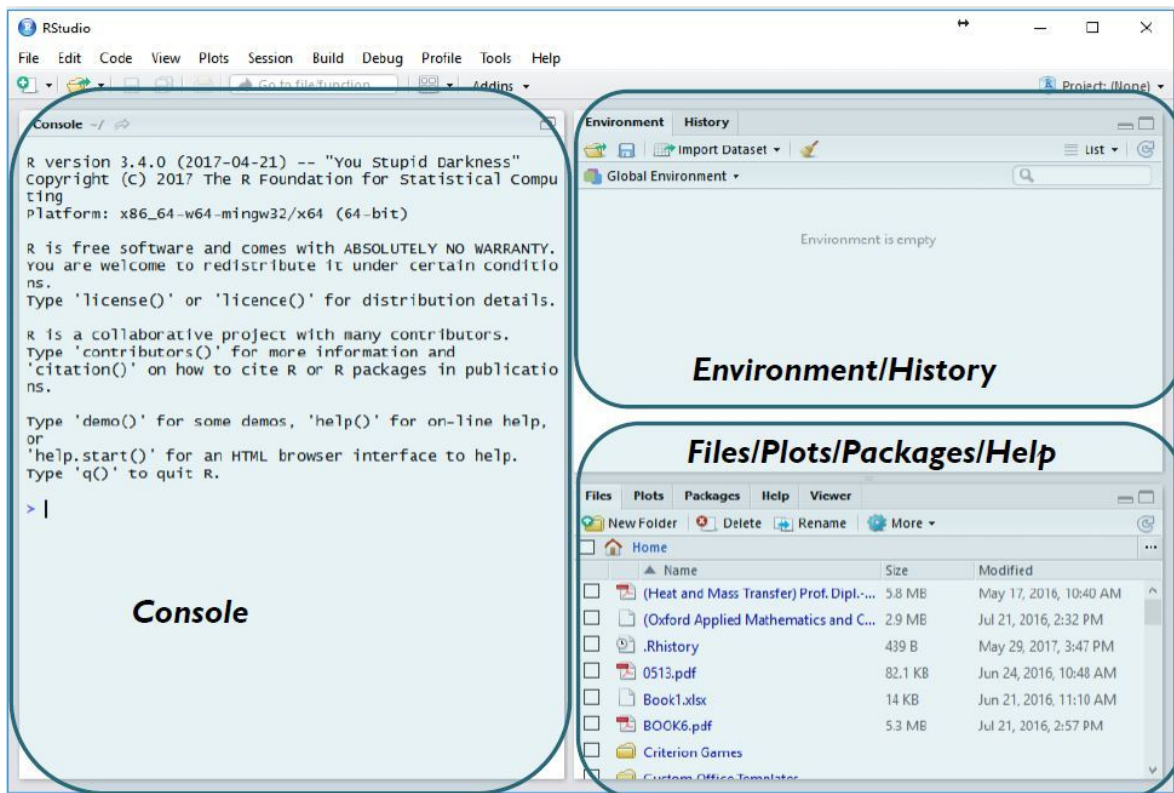


Figure 2.1: Interfaz de RStudio, un entorno de desarrollo integrado para R. Créditos a “Geeks-ForGeeks”

Arriba pueden observar lo que es una interfaz de RStudio. La ventana de la izquierda es la consola. La consola es donde se ejecutan las instrucciones que escribimos en el script. Por ejemplo, si queremos cargar un paquete, tenemos que escribir `library(nombre_del_paquete)` en el script y luego ejecutar esa línea de código para que se cargue el paquete en la consola.

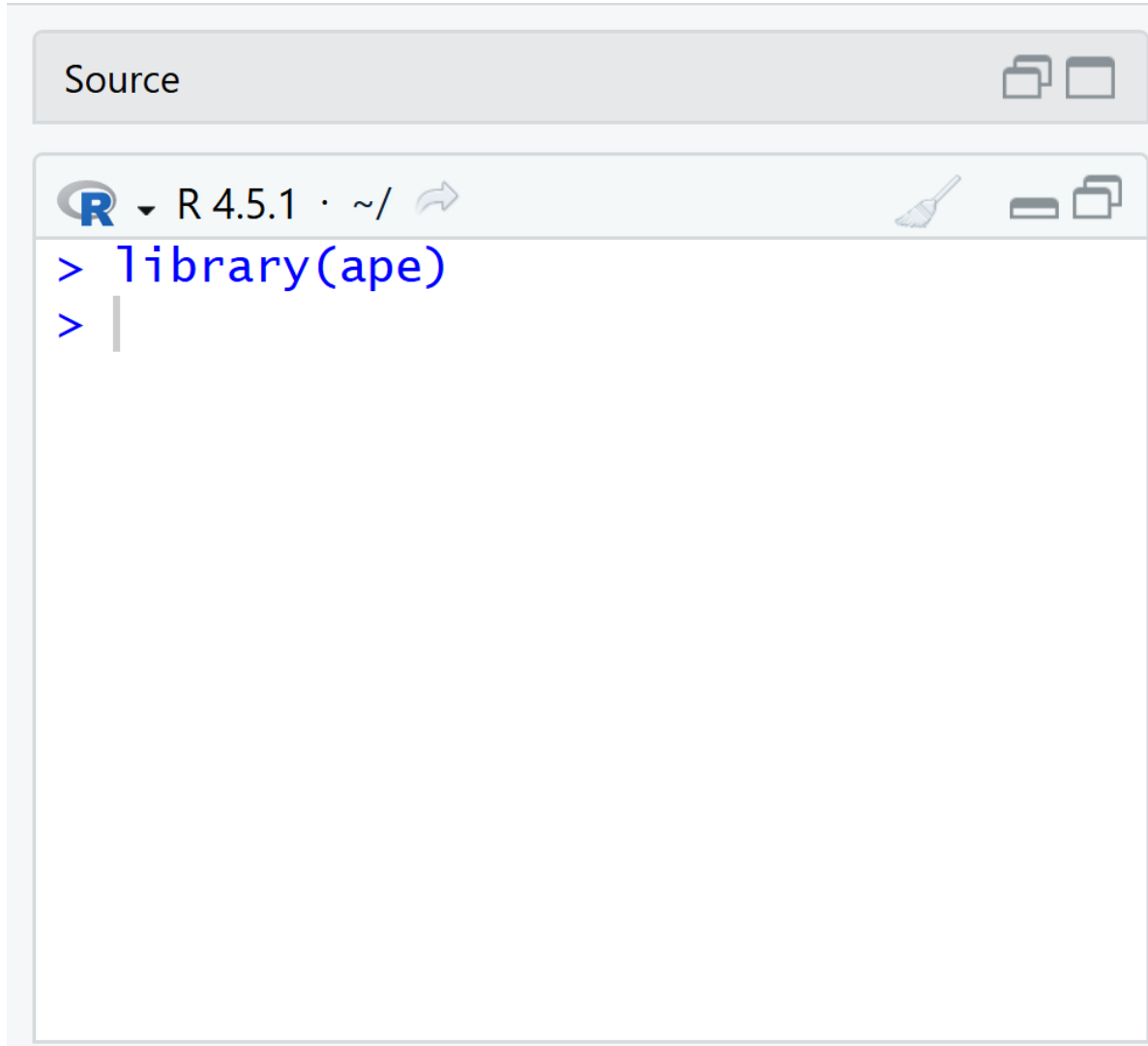


Figure 2.2: Ejecución de una instrucción en RStudio.

Como notarás, no se ejecuta nada más. !Y eso está bien!. No prestemos por ahora atención sobre los mensajes que aparecen en la consola. Lo importante es que se ejecute la instrucción que escribimos en el script. **En este caso, lo que hicimos fue cargar el paquete ape, que es un paquete muy útil para trabajar con datos de secuencias de ADN.**

A partir de esta sección, ya no verás imágenes de la consola de RStudio ejecutándose, sino que ahora verás el código tal cuál debe escribirse en tu consola. Esto es para que puedas copiar y pegar el código directamente en tu consola de RStudio y así practicar lo que estás aprendiendo.

Lo siguiente que haremos es generar datos para que R pueda leerlos. Los datos en R se clasifican en diferentes tipos: *vectores*, *matrices*, *data frames* y *listas*.

3 Vectores

Los vectores son la forma más básica de datos en R. Un vector es *una secuencia de elementos del mismo tipo*. Por ejemplo, un vector de números enteros, un vector de caracteres, etc. Para crear un vector en R, usamos la función `c()`, que significa “combine” o “concatenate”. Por ejemplo, para crear un vector de números enteros del 1 al 5, escribimos:

```
mi_vector <- c(1, 2, 3, 4, 5)
```

En este caso, `mi_vector` es un vector que contiene los números del 1 al 5. Podemos acceder a los elementos de un vector usando índices. Por ejemplo, para acceder al tercer elemento de `mi_vector`, escribimos:

```
mi_vector[3]
```

```
[1] 3
```

Esto nos devolverá el número 3, que es el tercer elemento del vector. También podemos realizar operaciones con vectores. Por ejemplo, si queremos sumar 2 a cada elemento del vector, escribimos:

```
mi_vector + 2
```

```
[1] 3 4 5 6 7
```

Esto nos devolverá un nuevo vector con los valores 3, 4, 5, 6 y 7.

Nota: los vectores son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en un vector para facilitar su análisis. Por ejemplo:

```
secuencias <- c("ATCG", "GCTA", "TACG", "CGTA")
```

En este caso, `secuencias` es un vector que contiene las secuencias de ADN. Cada elemento del vector es una secuencia de ADN representada como una cadena de caracteres. Podemos acceder a cada secuencia usando índices, por ejemplo:

4 Matrices

Las matrices son una extensión de los vectores. Una matriz es *una colección de elementos del mismo tipo organizados en filas y columnas*. Para crear una matriz en R, usamos la función `matrix()`. Por ejemplo, para crear una matriz de 2 filas y 3 columnas con los números del 1 al 6, escribimos:

```
mi_matriz <- matrix(1:6, nrow = 2, ncol = 3)
```

En este caso, `mi_matriz` es una matriz que contiene los números del 1 al 6 organizados en 2 filas y 3 columnas. Podemos acceder a los elementos de una matriz usando índices de fila y columna. Por ejemplo, para acceder al elemento en la primera fila y segunda columna, escribimos:

```
mi_matriz[1, 2]
```

```
[1] 3
```

Esto nos devolverá el número 4, que es el elemento en la primera fila y segunda columna de la matriz. También podemos realizar operaciones con matrices. Por ejemplo, si queremos sumar 2 a cada elemento de la matriz, escribimos:

```
mi_matriz + 2
```

```
      [,1] [,2] [,3]  
[1,]    3    5    7  
[2,]    4    6    8
```

Esto nos devolverá una nueva matriz con los valores 3, 4, 5, 6, 7 y 8.

Nota: las matrices son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en una matriz para facilitar su análisis. Por ejemplo:

```
secuencias_matriz <- matrix(c("ATCG", "GCTA", "TACG", "CGTA"), nrow = 2, ncol = 2)
secuencias_matriz
```

```
      [,1] [,2]
[1,] "ATCG" "TACG"
[2,] "GCTA" "CGTA"
```

En este caso, `secuencias_matriz` es una matriz que contiene las secuencias de ADN organizadas en 2 filas y 2 columnas. Cada elemento de la matriz es una secuencia de ADN representada como una cadena de caracteres.

5 Data frames

Los data frames son una estructura de datos muy común en R. Un data frame es *una tabla de datos que puede contener diferentes tipos de variables*. Para crear un data frame en R, usamos la función `data.frame()`. Por ejemplo, para crear un data frame con información sobre secuencias de ADN, escribimos:

```
secuencias_df <- data.frame(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_df` es un data frame que contiene información sobre las secuencias de ADN. La primera columna `id` contiene los identificadores de las secuencias, la segunda columna `secuencia` contiene las secuencias de ADN y la tercera columna `longitud` contiene la longitud de cada secuencia. Podemos acceder a las columnas de un data frame usando el operador `$`. Por ejemplo, para acceder a la columna `secuencia`, escribimos:

```
secuencias_df$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a las filas de un data frame usando índices. Por ejemplo, para acceder a la primera fila del data frame, escribimos: `wzjsecuencias_df[1, ]` Esto nos devolverá la primera fila del data frame, que contiene la información sobre la secuencia `seq1`. También podemos realizar operaciones con data frames. Por ejemplo, si queremos agregar una nueva columna que contenga el número de A's en cada secuencia, escribimos: `wzjsecuencias_df$num_A <- sapply(secuencias_df$secuencia, function(x) sum(str_count(x, "A")))` Esto nos agregará una nueva columna `num_A` al data frame que contiene el número de A's en cada secuencia de ADN.

Nota: los data frames son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar la información sobre las secuencias en un data frame para facilitar su análisis. Por ejemplo, podemos

agregar más columnas al data frame para incluir información adicional sobre las secuencias, como la especie a la que pertenecen, la ubicación geográfica donde se encontraron, etc. Esto nos permitirá realizar análisis más complejos y obtener insights más profundos sobre nuestras secuencias de ADN.

6 Listas

Las listas son una estructura de datos muy flexible en R. Una lista es *una colección de elementos que pueden ser de diferentes tipos*. Para crear una lista en R, usamos la función `list()`. Por ejemplo, para crear una lista que contenga información sobre secuencias de ADN, escribimos:

```
secuencias_lista <- list(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_lista` es una lista que contiene información sobre las secuencias de ADN. La lista tiene tres elementos: `id`, `secuencia` y `longitud`. Cada elemento de la lista es un vector que contiene la información correspondiente. Podemos acceder a los elementos de una lista usando el operador `$`. Por ejemplo, para acceder al elemento `secuencia`, escribimos:

```
secuencias_lista$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a los elementos de una lista usando índices. Por ejemplo, para acceder al primer elemento de la lista, escribimos:

```
secuencias_lista[[1]]
```

```
[1] "seq1" "seq2" "seq3" "seq4"
```

Esto nos devolverá el vector `id` que contiene los identificadores de las secuencias. Las listas son muy útiles cuando queremos almacenar información que no necesariamente tiene la misma estructura, como por ejemplo, una lista de secuencias de ADN donde cada secuencia puede tener una longitud diferente. Por ejemplo, podemos crear una lista de secuencias de ADN donde cada secuencia es un vector de caracteres de diferente longitud:

```
secuencias_lista <- list(  
  seq1 = c("A", "T", "C", "G"),  
  seq2 = c("G", "C", "T", "A"),  
  seq3 = c("T", "A", "C"),  
  seq4 = c("C", "G")  
)
```

En este caso, **secuencias_lista** es una lista que contiene cuatro elementos, cada uno de los cuales es un vector de caracteres que representa una secuencia de ADN. Cada secuencia tiene una longitud diferente, lo que demuestra la flexibilidad de las listas en R.

Con eso concluye esta sección introductoria sobre R para principiantes. En las siguientes secciones, aprenderemos a usar R para realizar análisis más complejos con nuestros datos de secuencias de ADN mitocondrial.

7 Trabajos en el Laboratorio I

En el laboratorio de Antropología Molecular, es común trabajar con datos de secuencias de ADN mitocondrial (mtDNA). En esta sección, aprenderemos a descargar datos desde GenBank y a leerlos en R para su posterior análisis.

Nota: Asegúrate de tener listos tus archivos de secuencias en formato FASTA antes de comenzar. Si no los tienes, puedes continuar siguiendo las instrucciones que daré a continuación.

7.1 1. Descarga de datos desde GenBank

GenBank es una base de datos pública de secuencias de ADN mantenida por el NCBI [National Center for Biotechnology Information; Sayers et al. (2022)]. Para descargar datos de GenBank, puedes utilizar la función `rentrez` en R o hacerlo manualmente a través de la página web del NCBI. Para descargar los datos manualmente, naveguen a la página de NCBI y seleccionen el formato FASTA como se muestra a continuación:

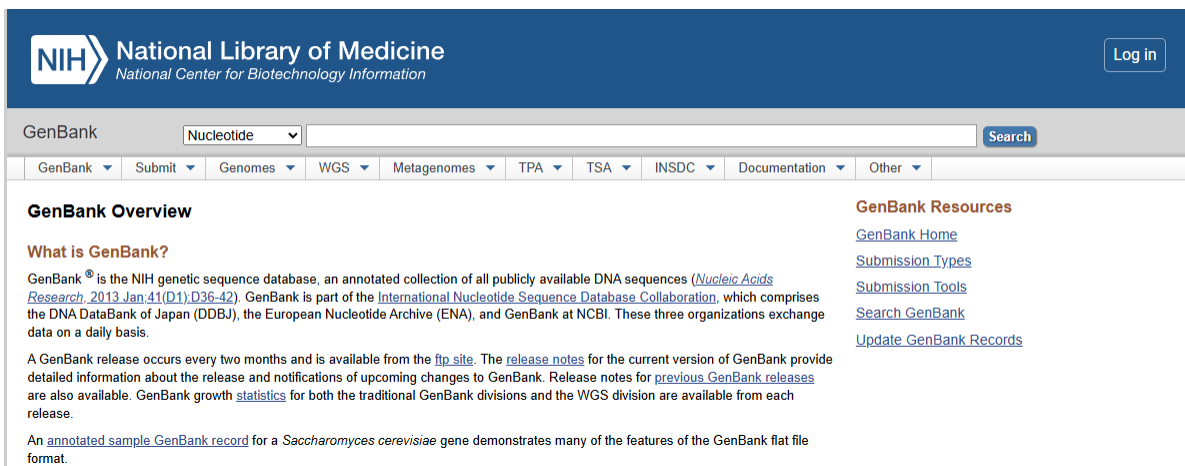


Figure 7.1: Página de inicio de NCBI. Las opciones por default se muestran en la imagen

Lo primero que debemos hacer es buscar una secuencia de interés. Para esto, podemos utilizar el número de acceso (accession number) o el nombre del organismo. En este caso, decidí usar solo las secuencias de mtDNA de la región control, que es una región comúnmente utilizada en

estudios de genética de poblaciones. Para esto, busqué “Human mitochondrial DNA control region” en la barra de búsqueda y seleccioné las secuencias que me interesaban.

Nucleotide

Human mitochondrial DNA control region

Create alertAdvanced

Summary200 per pageSort by Default orderSend to:

Items: 1 to 200 of 11359

<< First < Prev Page 1 of 57 Next > Last >>

Filters activated: Sequence length from 1 to 400. Clear all

☐

[Human mitochondrial DNA control region..genotype AK1110](#)

1. 377 bp linear DNA
Accession: U37731.1 GI: 1022848
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AK27](#)

2. 377 bp linear DNA
Accession: U37730.1 GI: 1022847
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW222](#)

3. 377 bp linear DNA
Accession: U37733.1 GI: 1022850
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW218](#)

4. 377 bp linear DNA
Accession: U37732.1 GI: 1022849
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype HK3992](#)

5. 377 bp linear DNA
Accession: U37737.1 GI: 1022854
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

Figure 7.2: Captura de pantalla de la búsqueda en NCBI. En la barra de búsqueda se muestra “HVR1 mtDNA”

Una vez que tenga identificadas las secuencias de interés, puedo descargarlas directamente, sin embargo, para este ejercicio, usaremos el paquete **ape** para descargar las secuencias directamente desde R, lo que nos permitirá automatizar el proceso y evitar errores manuales. Para ello, debo identificar el número de acceso de la secuencia que quiero descargar. En este caso, usaré la secuencia con el número de acceso “U37731.1”, que corresponde a una secuencia de mtDNA humana de la región control.

```
#Cargamos el paquete "ape"
library(ape)

#Descargamos la secuencia de interés utilizando su número de acceso
secuencia_1 <- read.GenBank("U37731.1")

# Visualizamos qué descargamos
print(secuencia_1)
```

Sin embargo, es necesario que se descarguen al menos un número de secuencias para que el código funcione correctamente. Para esto, podemos descargar varias secuencias utilizando sus números de acceso. Por ejemplo, podemos descargar las siguientes secuencias:

```
# Descargamos varias secuencias utilizando sus números de acceso
secuencias <- read.GenBank(c("U37731.1", "U37730.1", "U37733.1", "U37737.1", "U37752.1",
                             "U37751.1", "U37739.1", "U37743.1"))

# Visualizamos las secuencias descargadas
print(secuencias)
```

8 Summary

In summary, this book has no content whatsoever.

References

(**article?**) {Sayers2022, abstract = {GenBank® (<https://www.ncbi.nlm.nih.gov/genbank/>) is a comprehensive, public database that contains 15.3 trillion base pairs from over 2.5 billion nucleotide sequences for 504 000 formally described species. Recent updates include resources for data from the SARS-CoV-2 virus, including a SARS-CoV-2 landing page, NCBI Datasets, NCBI Virus and the Submission Portal. We also discuss upcoming changes to GI identifiers, a new data management interface for BioProject, and advice for providing contextual metadata in submissions.}, author = {Eric W Sayers and Mark Cavanaugh and Karen Clark and Kim D Pruitt and Conrad L Schoch and Stephen T Sherry and Ilene Karsch-Mizrachi}, doi = {10.1093/nar/gkab1135}, issn = {0305-1048}, issue = {D1}, journal = {Nucleic Acids Research}, month = {1}, pages = {D161-D164}, title = {GenBank}, volume = {50}, url = {<https://doi.org/10.1093/nar/gkab1135>}, year = {2022} }

Sayers, Eric W, Mark Cavanaugh, Karen Clark, Kim D Pruitt, Conrad L Schoch, Stephen T Sherry, and Ilene Karsch-Mizrachi. 2022. “GenBank.” *Nucleic Acids Research* 50 (January): D161–64. <https://doi.org/10.1093/nar/gkab1135>.